

Day 17: Inheritance & Hardware Abstraction Layer

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

Base Classes and Derived Classes

Inheritance allows a derived class to share interface and/or implementation with a base class. In embedded, this is used to create hardware abstraction layers:

```
class ISensor { /* abstract interface */ };          // Base
class Bme280 : public ISensor { /* ... */ };        // Derived – concrete implementation
class FakeSensor : public ISensor { /* ... */ };    // Derived – test double
```

Abstract Base Class with Pure Virtual

A class with at least one **pure virtual** function (`= 0`) cannot be instantiated directly — it defines an interface that derived classes must implement:

```
class ISensor {
public:
    virtual ~ISensor() = default;                // Virtual destructor – ALWAYS needed
    virtual bool    init()                        = 0;
    virtual bool    read(int32_t *out)            = 0;
    virtual const char *name() const              = 0;
};
```

Any class that inherits `ISensor` must implement all three functions, or it's also abstract.

Virtual Function vtable Overhead

Every class with virtual functions gets a **vtable** (virtual function table) — a static array of function pointers. Every instance of that class gets a **4-byte vtable pointer**:

- Memory cost: 4 bytes per object + one vtable per class (in Flash)
- Runtime cost: one extra pointer dereference per virtual call
- Code size cost: vtable + RTTI data (unless `-fno-rtti`)

For a small embedded system with 4 sensor objects, this is 16 bytes extra RAM — typically acceptable. For 1000 objects, reconsider.

Zephyr Driver Model Analogy

Zephyr's internal driver model uses a `device` struct with a `device_driver_api` struct — a C-style vtable:

```
struct sensor_driver_api {
    int (*attr_set)(const struct device *, enum sensor_channel, ...);
    int (*attr_get)(const struct device *, enum sensor_channel, ...);
    int (*sample_fetch)(const struct device *, enum sensor_channel);
    int (*channel_get)(const struct device *, enum sensor_channel, struct sensor_value *);
};
```

This is exactly what C++ virtual functions implement under the hood — Zephyr just does it in C.

HAL Design Pattern

A Hardware Abstraction Layer (HAL) separates:

1. **Interface** (`ISensor`) — what capabilities a sensor provides
2. **Driver** (`Bme280Sensor`) — how a specific hardware device provides them
3. **Application** — uses only the interface, knows nothing about the hardware

This lets you swap hardware (swap Bme280 for a different sensor) without changing application code, and use fake sensors for unit testing.

Interface Classes in Embedded

Keep interfaces minimal — only the operations the application actually needs:

```
class IComm {
public:
    virtual bool send(const uint8_t *data, size_t len) = 0;
    virtual bool recv(uint8_t *buf, size_t len, uint32_t timeout_ms) = 0;
};
// Implementations: UartComm, SpiComm, BleComm, UsbComm, FakeComm(for testing)
```

Practice Tasks

1. Add a third sensor type (`FakePressureSensor`) to the hierarchy and poll all three with a single `ISensor *` array
2. Measure the vtable pointer overhead: `sizeof(Bme280Sensor)` vs `sizeof(Bme280SensorNonVirtual)`
3. Implement `IActuator` interface with `set(uint8_t value)` — derive `PwmLed` and `RelayDriver`
4. Replace the virtual destructor with a non-virtual one — observe the potential memory leak with Valgrind or ASan on host

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day17
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day17/main.cpp](#)